

Entregable 2: Manual Operativo de Datos

1. Esquema de bases de datos (Notion)

El sistema usa 3 bases de datos vinculadas por relaciones, evitando datos aislados.

Base: Clientes

Campo	Tipo	Descripción
ID Cliente	Título	Nombre/empresa del cliente
Contacto	Email	Identificador único usado para buscar/evitar duplicados
Propuestas	Relación → Propuestas	Lado inverso: todas las propuestas de ese cliente

Base: Propuestas (núcleo del sistema)

Campo	Tipo	Descripción
ID Propuesta	Título	Nombre + timestamp, generado automáticamente
Cliente	Relación → Clientes	Cliente que solicitó la propuesta
Descripción original	Texto largo	Texto del mail, extraído sin modificar
Urgencia	Select (Alta/Media/Baja)	Clasificada por IA (Gemini Flash), define el ruteo Batch vs. tiempo real
Estado	Select	Pendiente → Procesado por IA → Esperando Aprobación → Aprobado por Humano → Enviado / Rechazado
Modelo usado	Select	Registro de qué modelo/estrategia procesó esta propuesta (trazabilidad de costos)
Propuesta generada (IA)	Texto largo	Output de la IA de redacción
Aprobado / Aprobador	Checkbox / Persona	Resultado del paso de Human-in-the-loop
Fecha de envío	Fecha	Se completa al enviar al cliente
Batch ID	Texto	ID del job de Batch API, solo para propuestas de Urgencia Media/Baja
Propuesta	Relación → Log de Errores	Lado inverso: errores asociados a esta propuesta

Base: Log de Errores

Campo	Tipo	Descripción
ID Error	Título	Identificador
Propuesta relacionada	Relación → Propuestas	A qué propuesta afecta el error
Tipo de error	Select	API IA falló / Datos incompletos / Timeout aprobación / Otro
Detalle	Texto largo	Mensaje de error crudo (<code>{{ \$json.error.message }}</code>)
Fecha	Fecha	Timestamp del fallo
Resuelto	Checkbox	Seguimiento manual

No es una base de datos sino una página de referencia, leída dinámicamente por el flujo (`Get page markdown`) en cada ejecución. Define las reglas de estilo, estructura y restricciones anti-alucinación que se inyectan como parte del prompt de sistema de la IA de redacción.

2. Esquemas JSON de transferencia entre integraciones

Gmail → Gemini Flash (extracción)

Entrada: cuerpo del mail en texto plano (`item.json.text`). Salida esperada del modelo:

```
{
  "nombre_empresa": "string",
  "email": "string",
  "descripcion": "string",
  "urgencia": "Alta | Media | Baja"
}
```

n8n → Notion (creación de Propuesta)

```
{
  "Descripción original": "string",
  "Urgencia": "select",
  "Estado": "Pendiente",
  "Cliente": [ "<page_id del cliente>" ]
}
```

Notion (manual de tono) → Gemini Pro (redacción)

El `system_instruction` combina el contenido markdown de la página del manual + reglas de estructura fijas. El `contents` (rol user) es la descripción original del cliente. Salida: texto libre (la propuesta redactada), sin estructura JSON forzada.

Diferimiento Media/Baja (implementación final)

Para las urgencias Media y Baja, el sistema no genera un esquema de transferencia distinto: el registro pasa por un nodo Wait (pausa hasta un horario de baja demanda) y luego **reutiliza exactamente el mismo esquema** de “Notion (manual de tono) → Gemini Pro” descrito arriba. No hay una segunda estructura de datos que documentar para este camino — es intencional, ya que reduce superficie de mantenimiento.

Diseño original (no implementado por restricciones de tiempo): se había definido un esquema de Batch API real, donde múltiples propuestas viajan juntas en un solo request con un campo `metadata.key` por cada una (el `page_id` de Notion), permitiendo identificar a qué registro corresponde cada resultado al recibir la respuesta del lote. Se documenta como posible iteración futura en el Entregable 3.

3. Sincronización entre fuentes

Como el sistema combina múltiples fuentes (email, Notion, dos modelos de IA), la sincronización no la gestiona ninguna herramienta por sí sola: la gestiona explícitamente el flujo de n8n, escribiendo y actualizando el campo `Estado` de Notion en cada paso, que actúa como única fuente de verdad sobre en qué punto del proceso está cada propuesta.